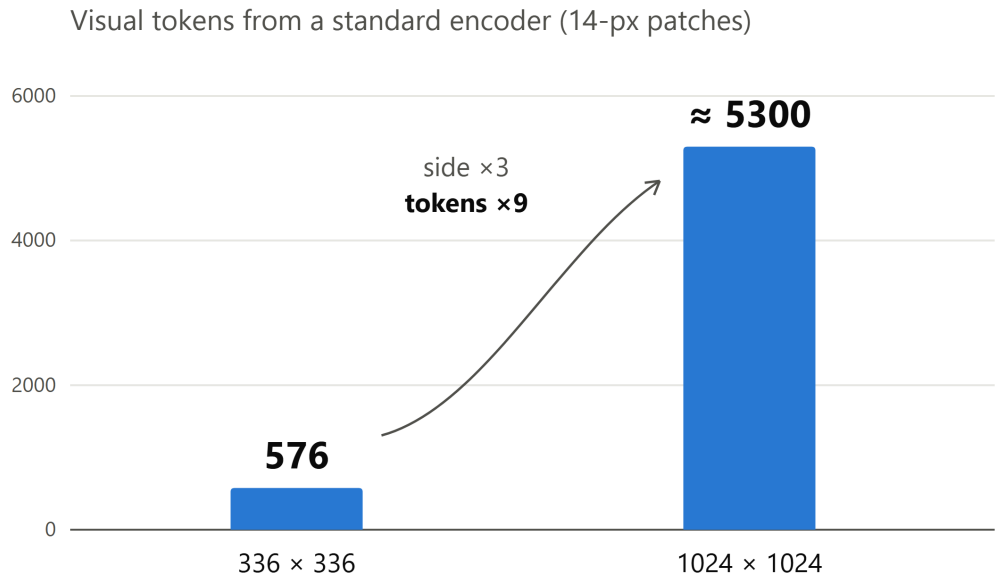
FastVLM

Redesigning the vision encoder for high resolution

CVPR 2025 · Apple

Higher resolution, slower VLM

TTFT = vision encoding time + LLM prefill time



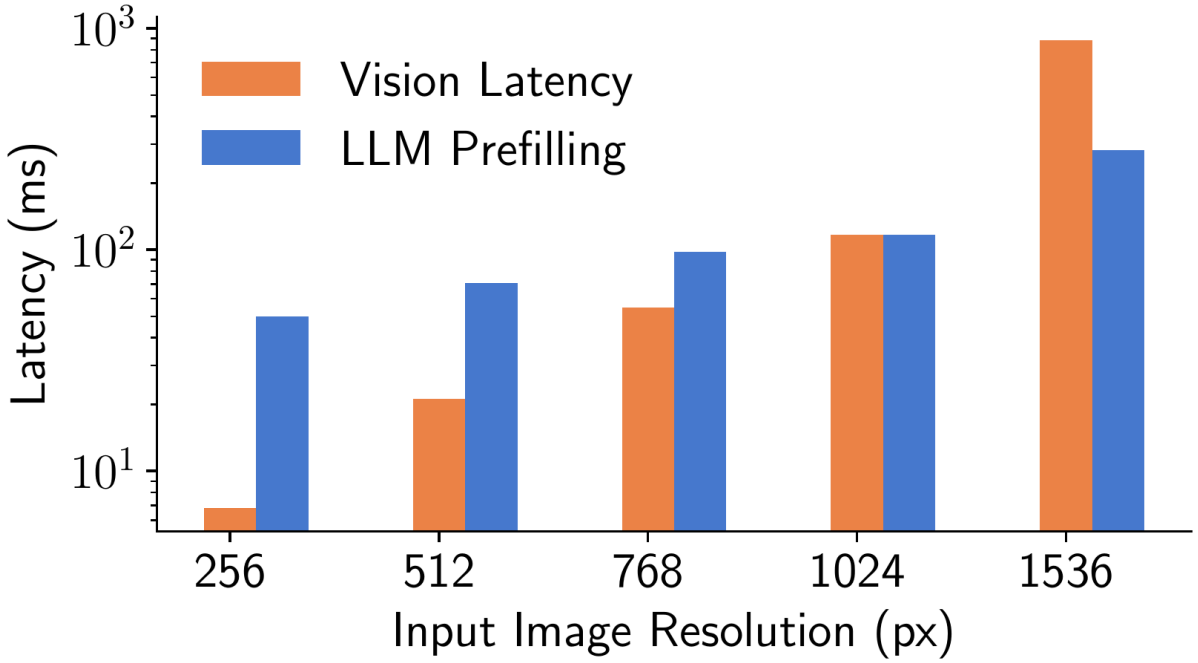
Token count grows with the **square** of resolution

More tokens → **both** parts get slower (e.g. self-attention: cost ∝ tokens^2)

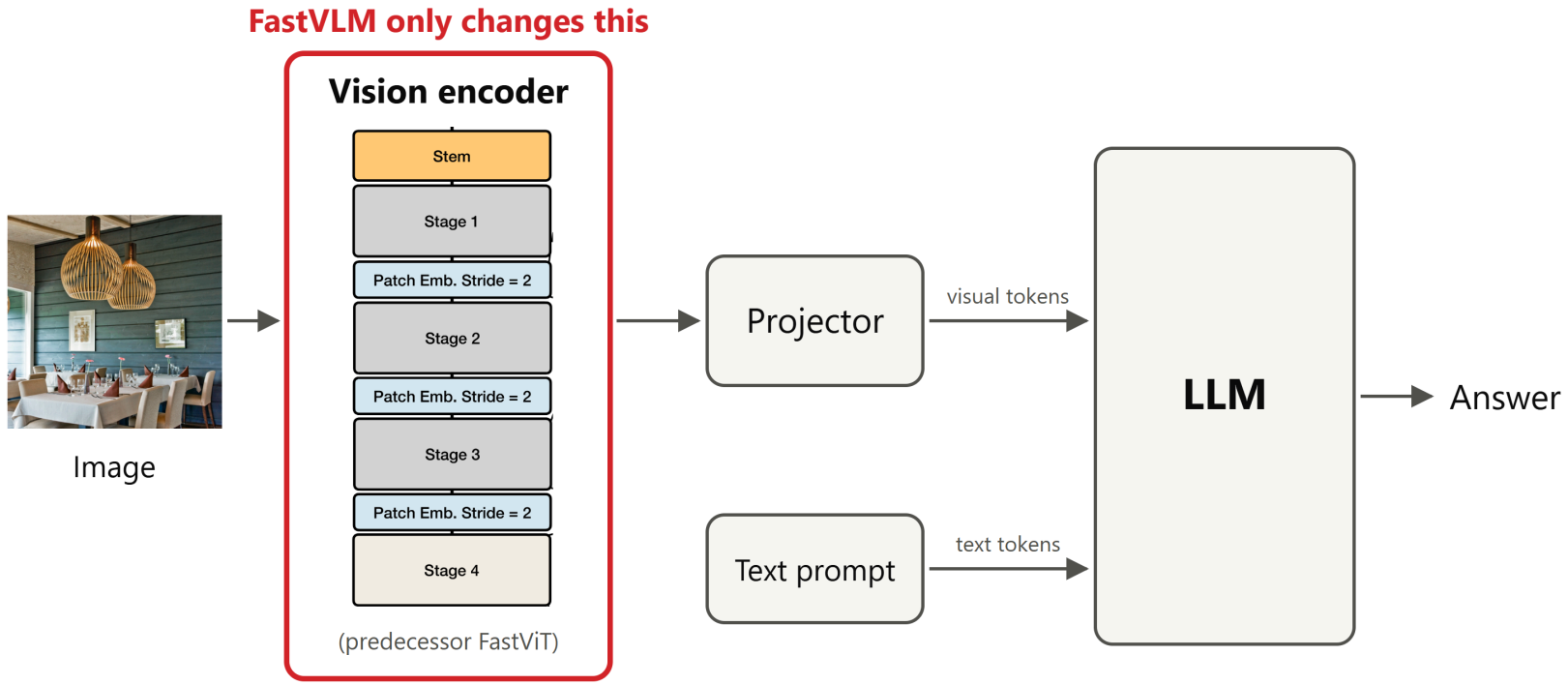
Trimming tokens is not enough

- At high resolution, **vision encoding** takes most of TTFT
- Trimming tokens after the encoder only shortens **prefill**: the encoder still processes the whole image

→ Can we use an encoder **built for high resolution**?

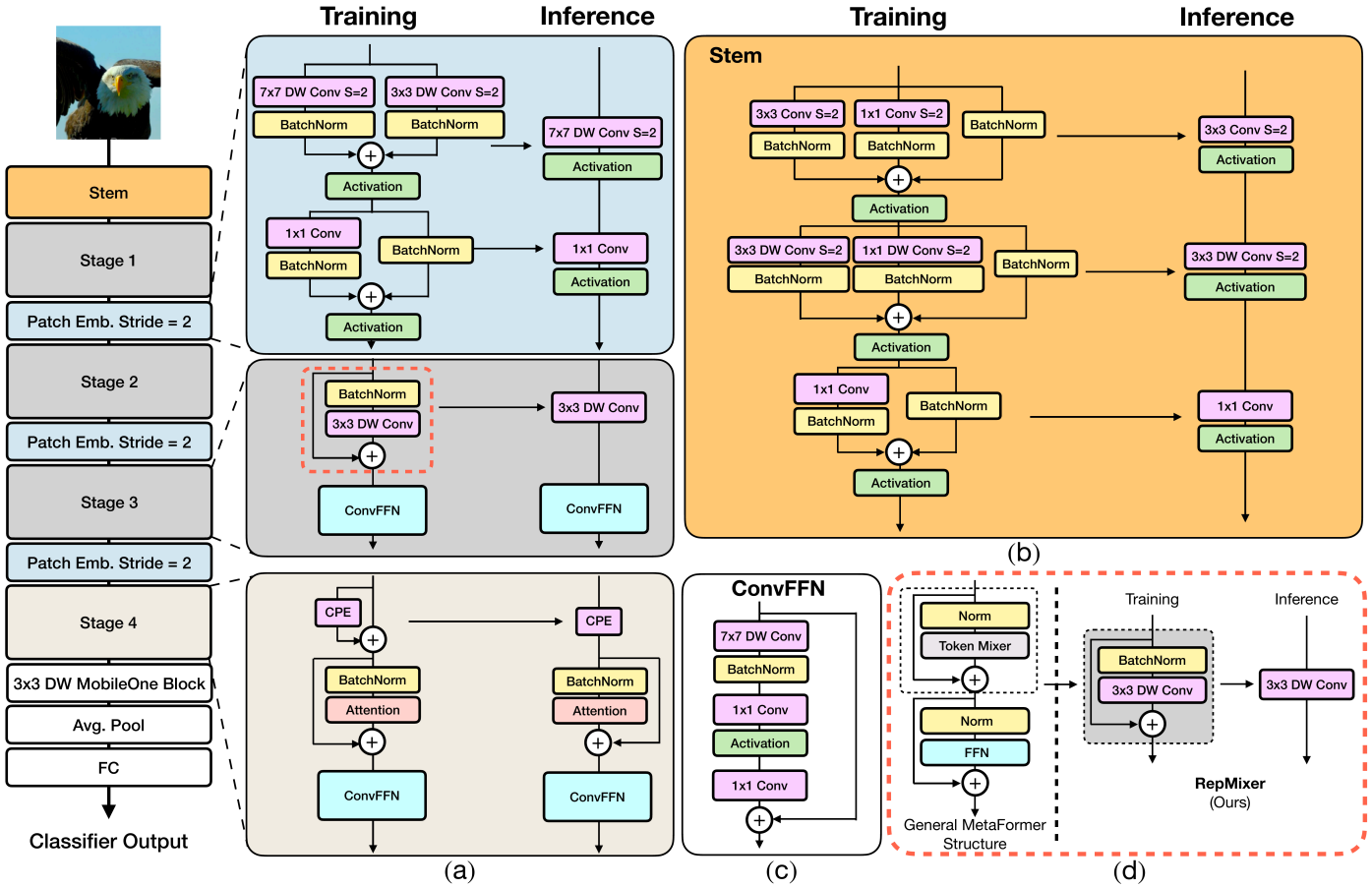


FastVLM swaps one box in the pipeline



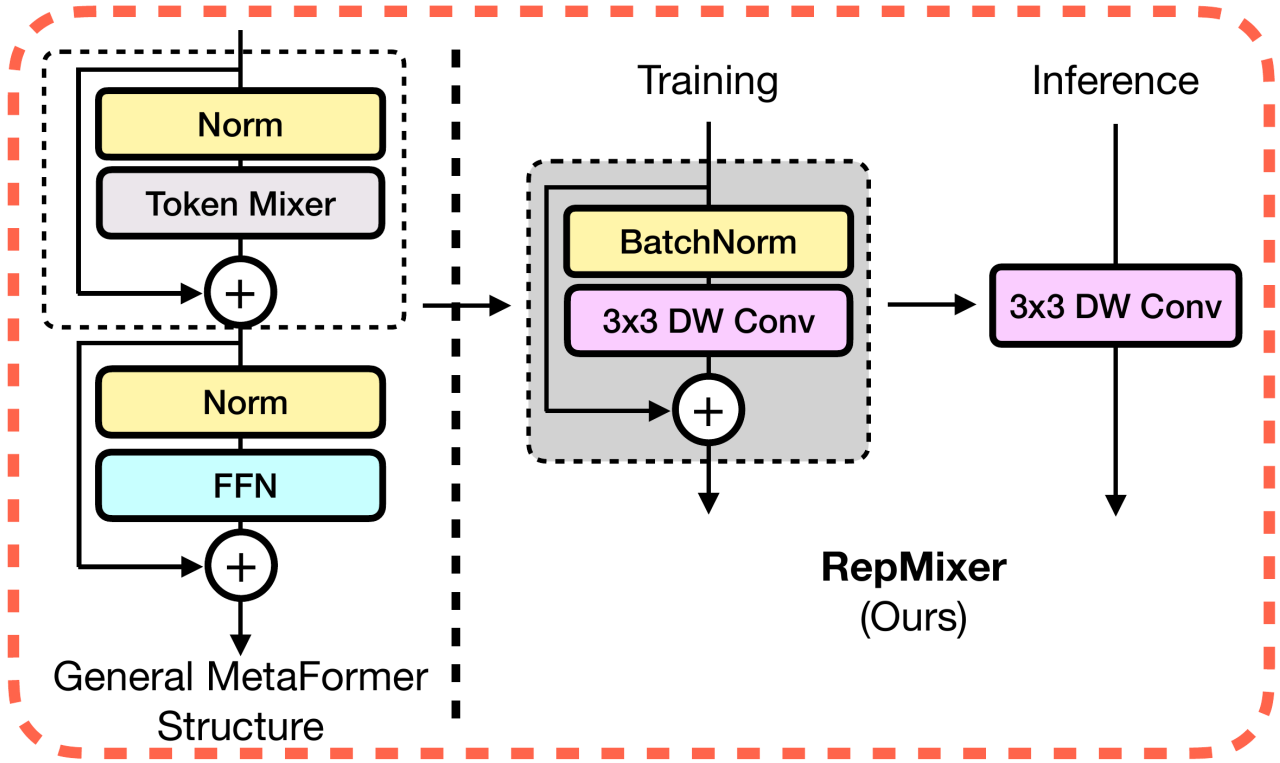
- Only the vision encoder changes; projector and LLM follow LLaVA
- Every stage from here on lives **inside** this box

Predecessor FastViT: the big picture



Left: the skeleton, a 4-stage pyramid | Right: the parts inside each block

The conv part is fine: one conv at inference



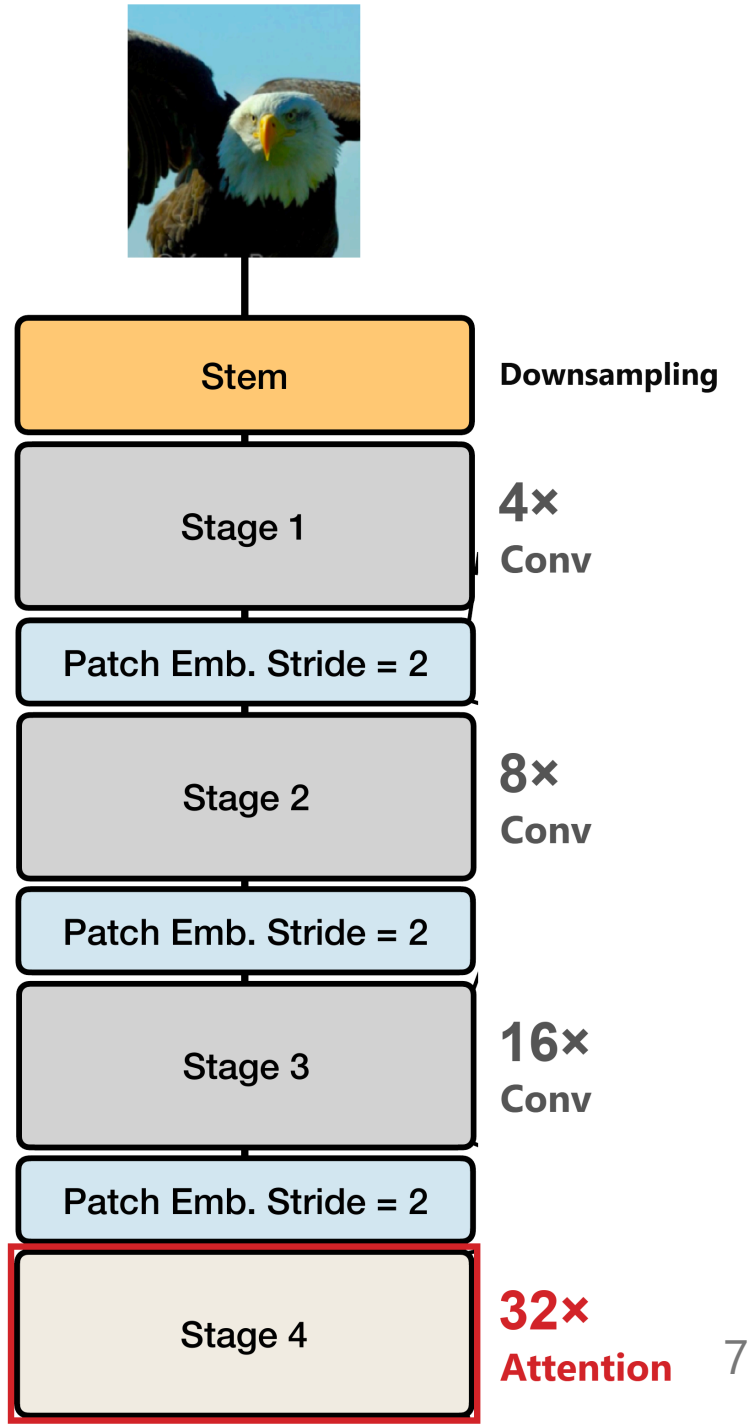
Training: several branches (easy to train) → Inference: merged into one conv (fast)

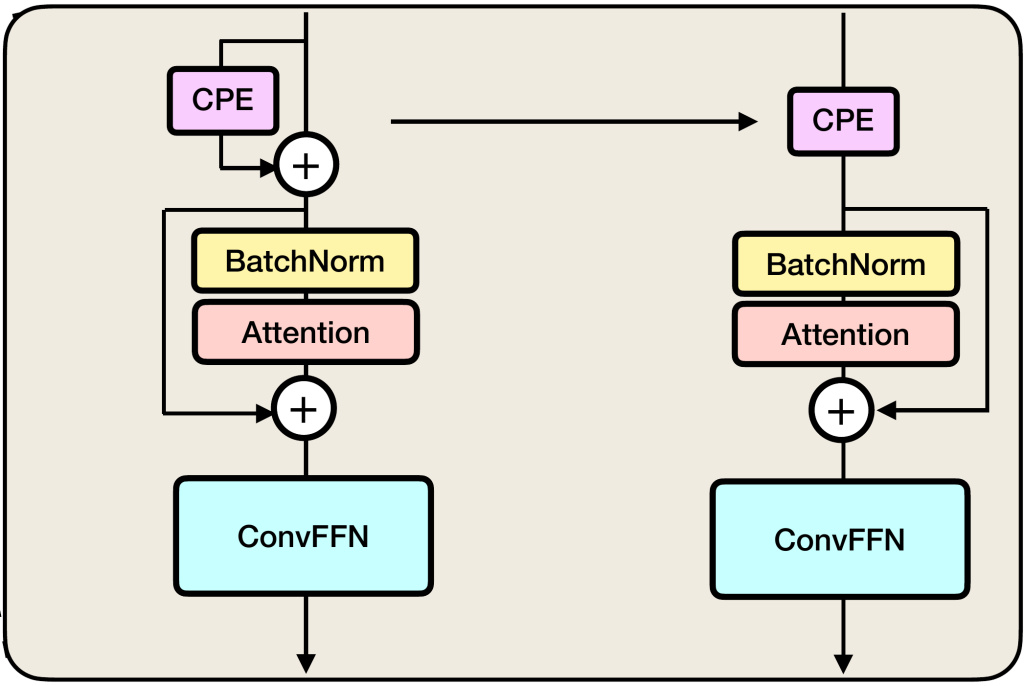
The bigger the image, the more it saves → **suits high resolution, keep as is**

Downsampled 32×: why is it still not enough?

- Each stage downsamples once; **every position left at the end is one token**
- 1024^2 image → 32×32 = **1024** tokens, still too many for the LLM

→ Can we **compress further**?



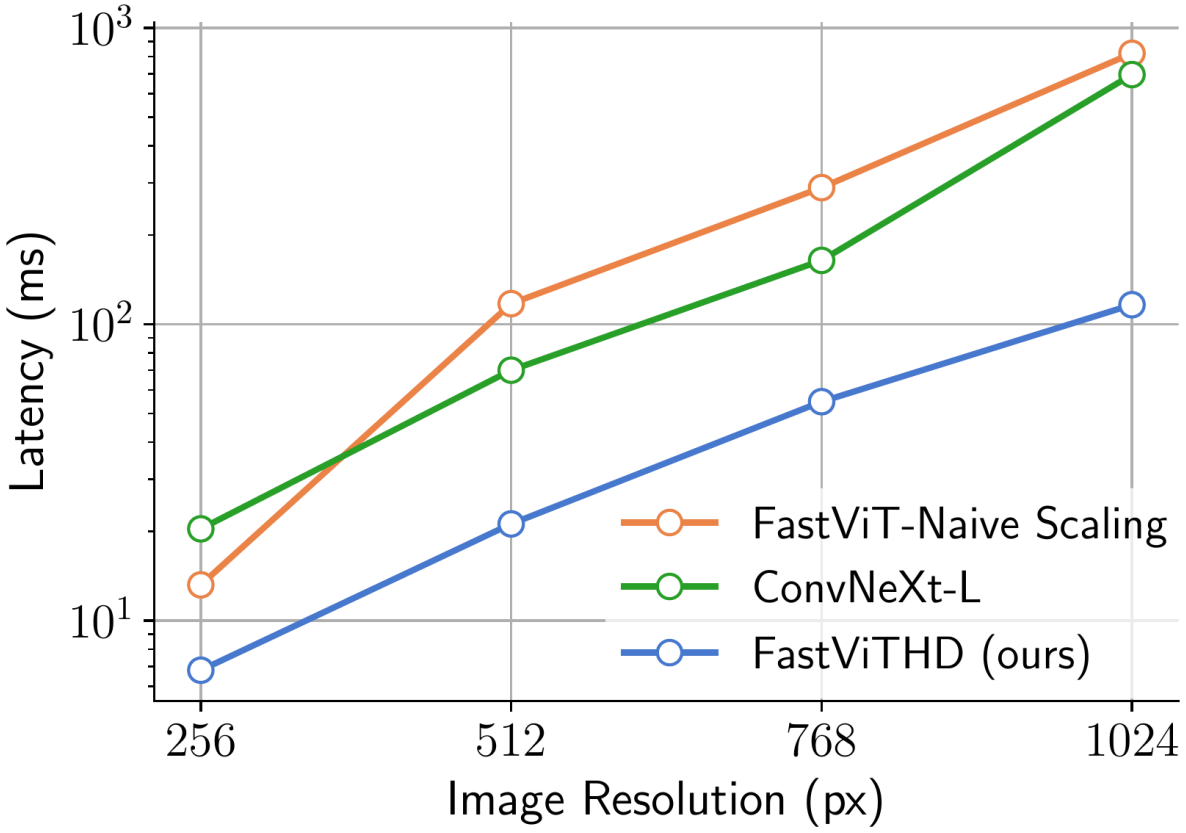


Self-attention comes too early

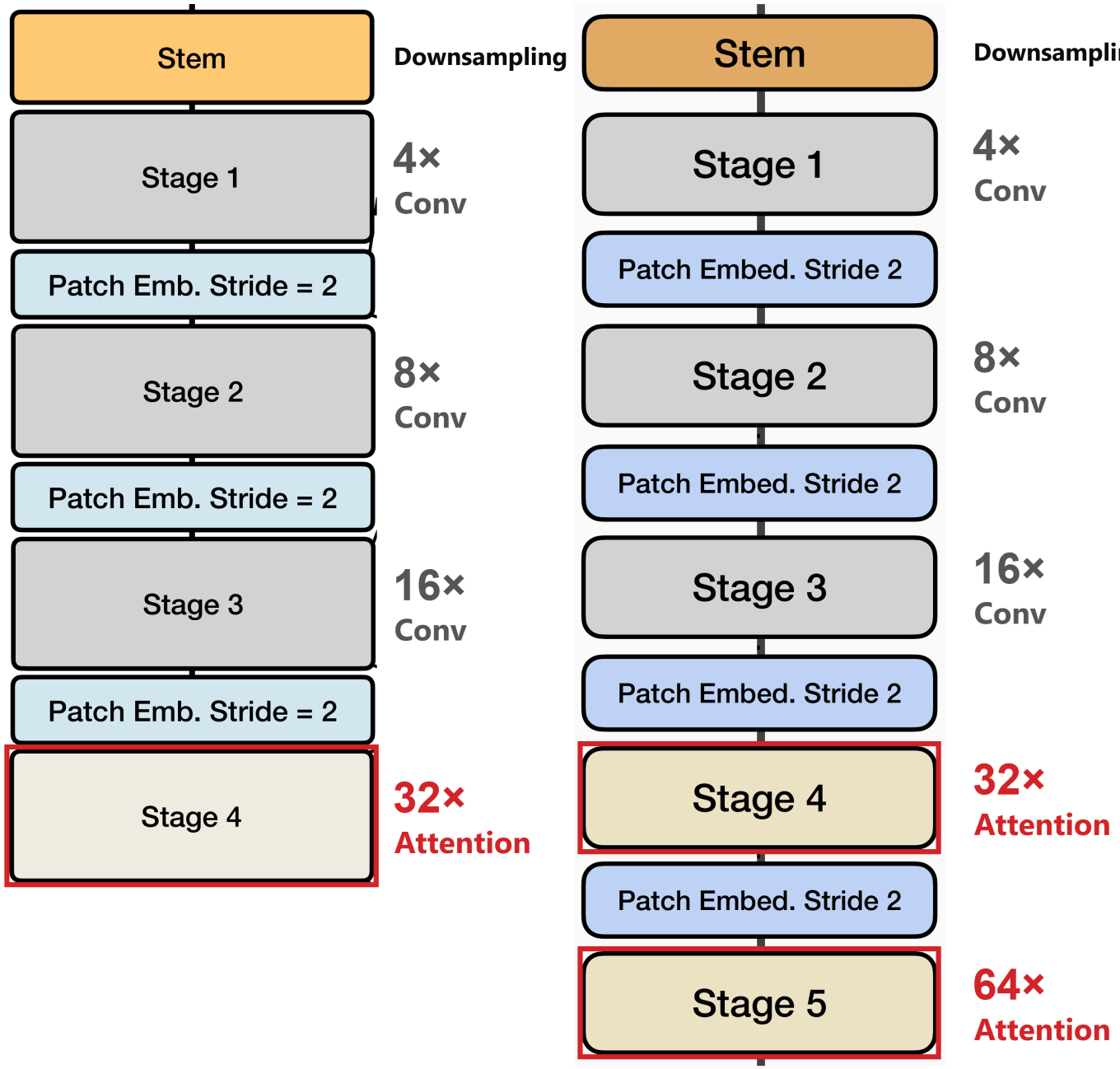
To scale a model up, the usual move is stacking self-attention in stages 3 and 4

Stage 3 (16× down)	Positions	Self-attention cost
224^2 image	196	cheap
1024^2 image	4096	about **400×** more

In practice: scaled the usual way, slower than pure conv



Orange: usual scaling | Green: pure-conv ConvNeXt-L | Blue: FastViTHD (next slide)



Compress one more level

- Add a 5th stage: 64× downsampling in total
- Move self-attention to the 32× and 64× stages

ViT	FastViT	FastViTHD
≈ 5300	1024	256

(tokens for a 1024^2 input)

Why didn't anyone do this earlier?

- The harder you compress, the more area one token covers → **small text gets lost**
- The view at the time: ViTamin recommends only 16× downsampling in total

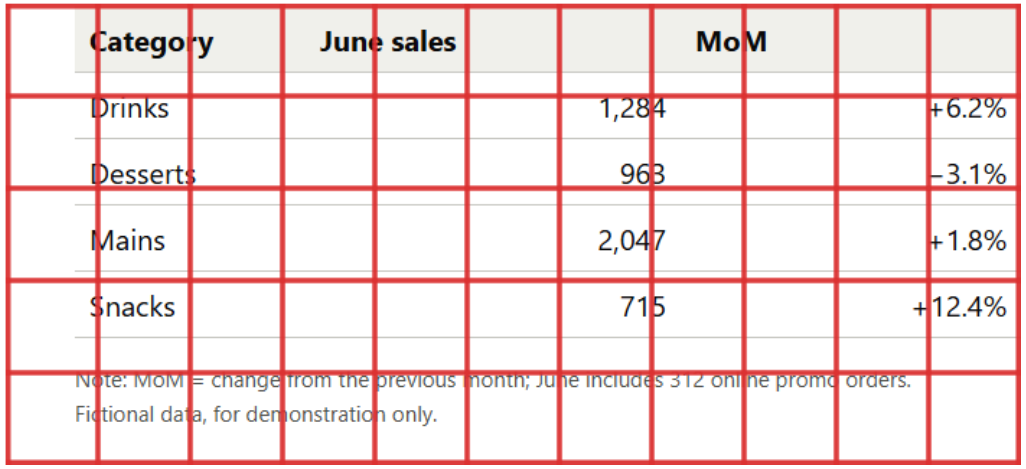
→ The real question: **how do we compress without losing information?**

32× downsampling: 1 cell = 32×32 px

Category	June sales	MoM
Drinks	1,284	+6.2%
Desserts	963	−3.1%
Mains	2,047	+1.8%
Snacks	715	+12.4%

Note: MoM = change from the previous month; June includes 312 online promo orders.
Fictional data, for demonstration only.

64× downsampling: 1 cell = 64×64 px

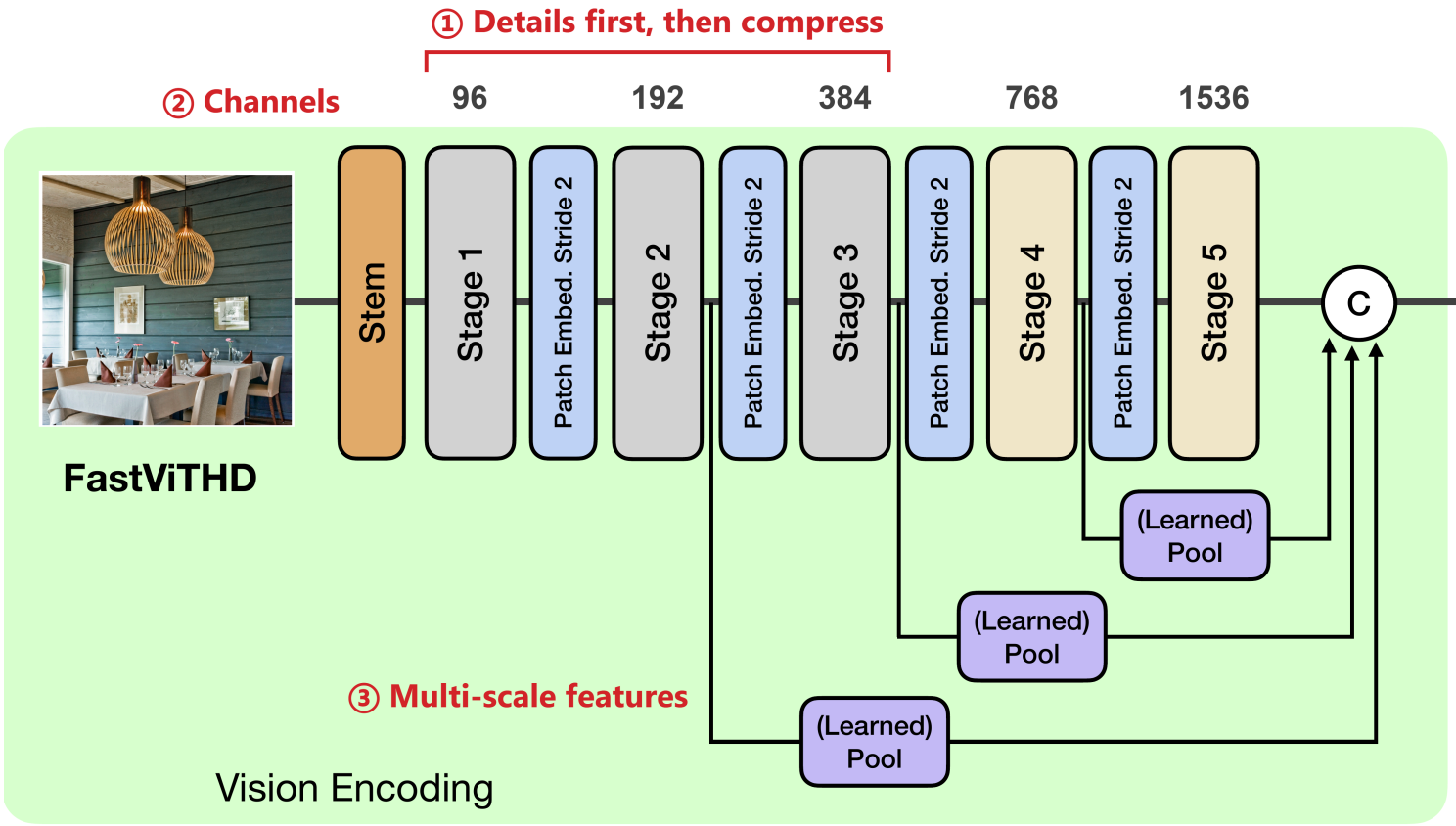


Compressing the right way

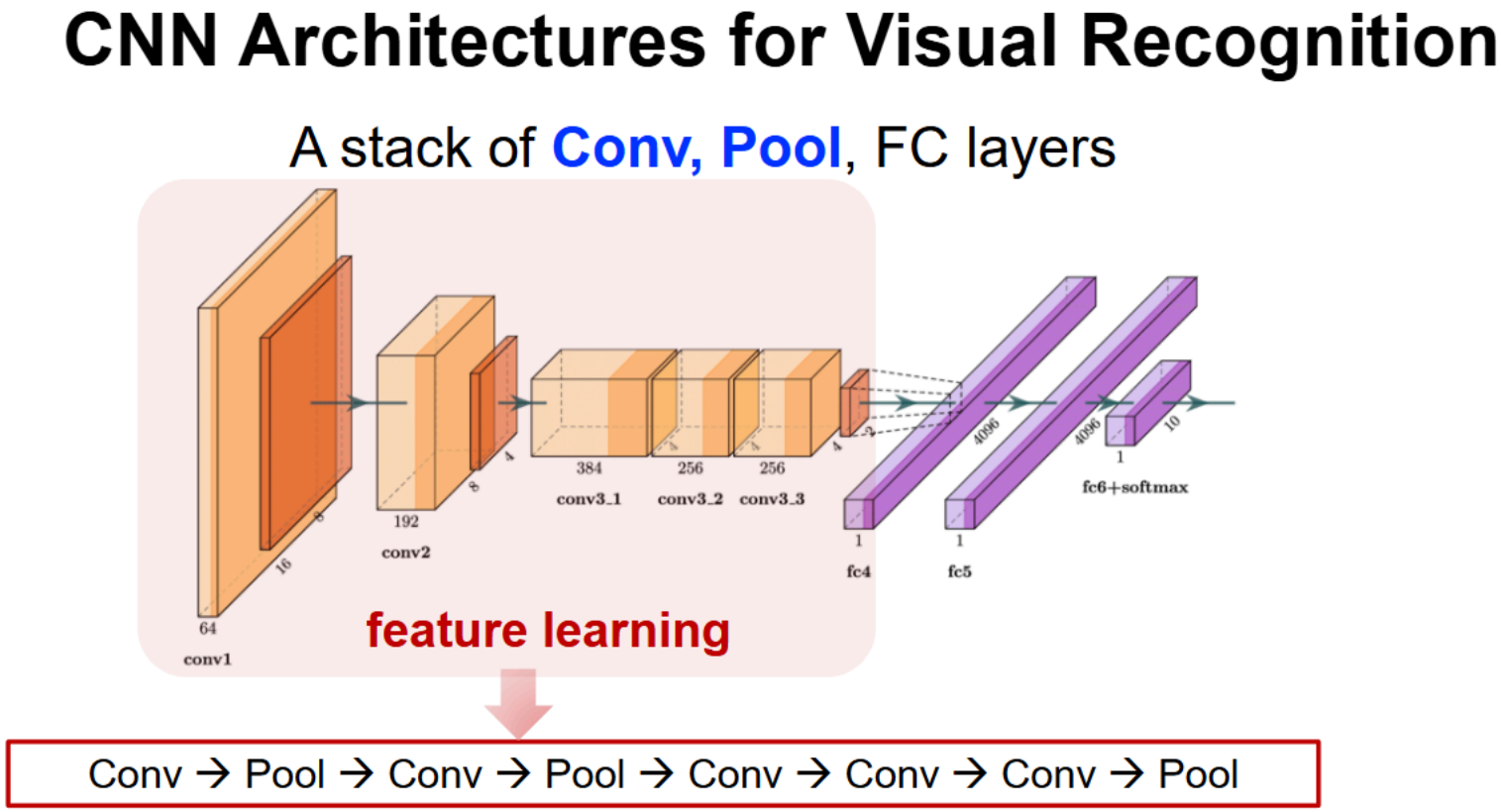
① Details first, then compress
Cheap convs (RepMixer) while the map is large; self-attention only once it is small

② Trade space for channels (next slide)

③ Multi-scale features (the slide after)



Remember this from class?

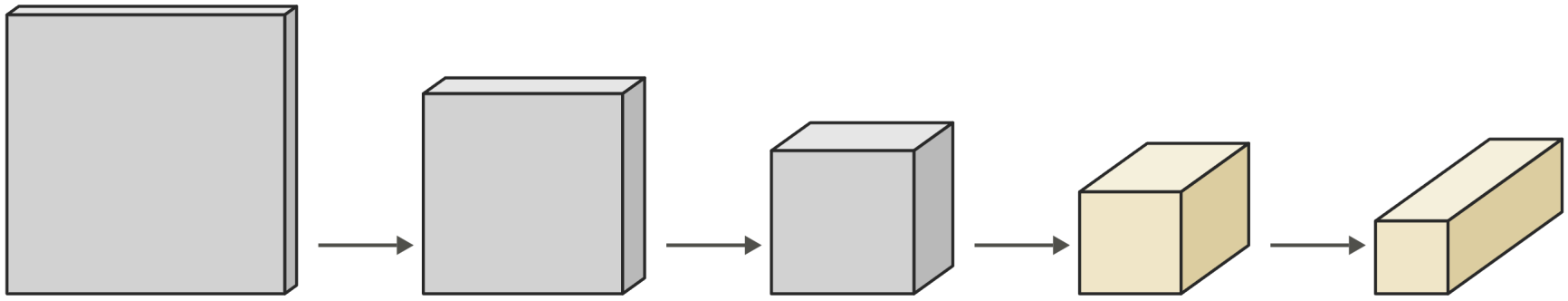


Each Conv / Pool step: the map gets **smaller**, the channels get **more** (64 → 192 → 384)

② Trading space for channels

- Same pattern in FastViTHD: 5 stages, channels grow from 96 to 1536
- The final 16×16 = 256 positions each carry **1536** numbers: fewer tokens, each holding more

Front face = height × width (positions) · Depth = channels



Stage 1
256×256 × 96

Stage 2
128×128 × 192

Stage 3
64×64 × 384

Stage 4
32×32 × 768

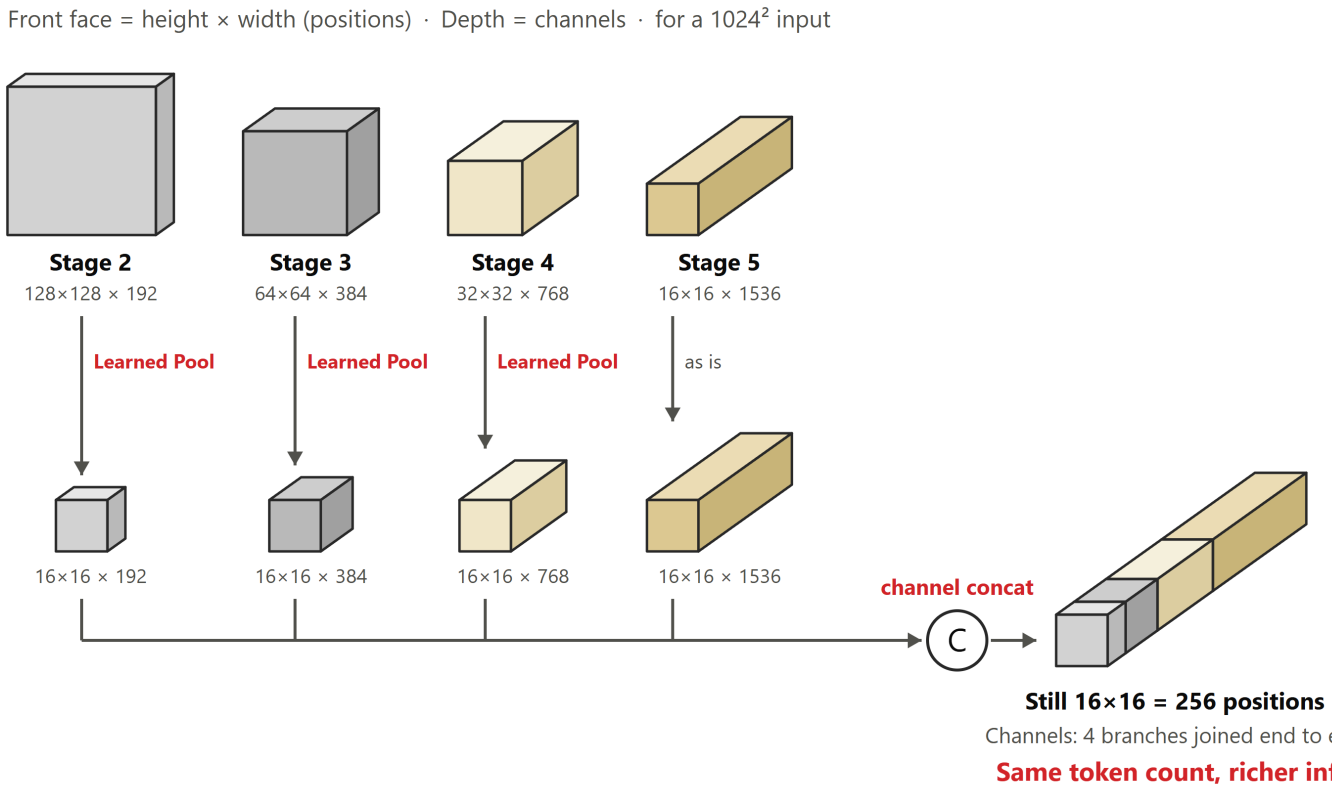
Stage 5
16×16 × 1536

Each downsampling: height and width halve, channels double → each position holds more

③ Multi-scale: what is the C?

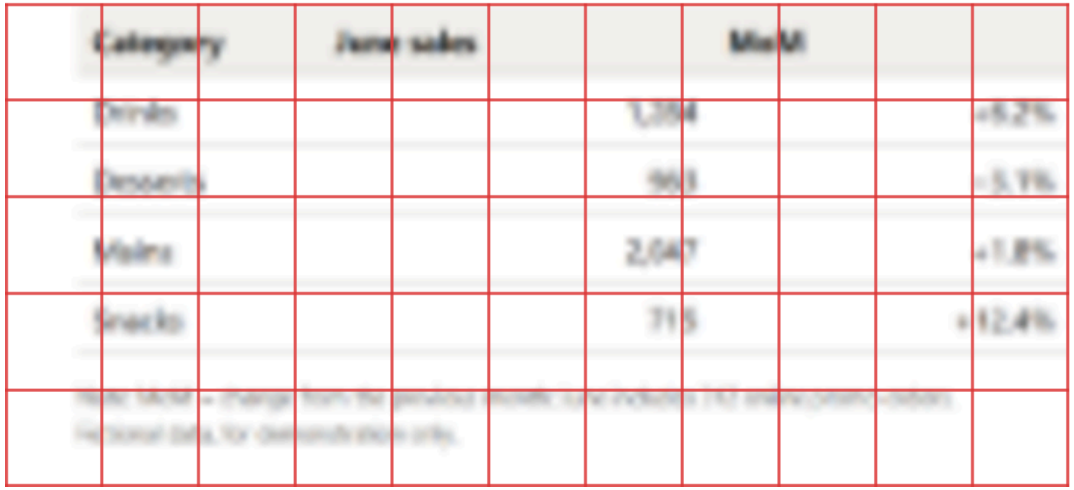
1. Take one branch each from stages 2, 3 and 4
2. **Learned Pool**: a strided depthwise conv shrinks each to stage 5's size (16×16)
3. **C**: concatenate the branches along the **channel** dimension

→ Still **256** tokens, but each token now carries both coarse and fine information



Compression buys resolution

ViT: 224×224, 256 tokens



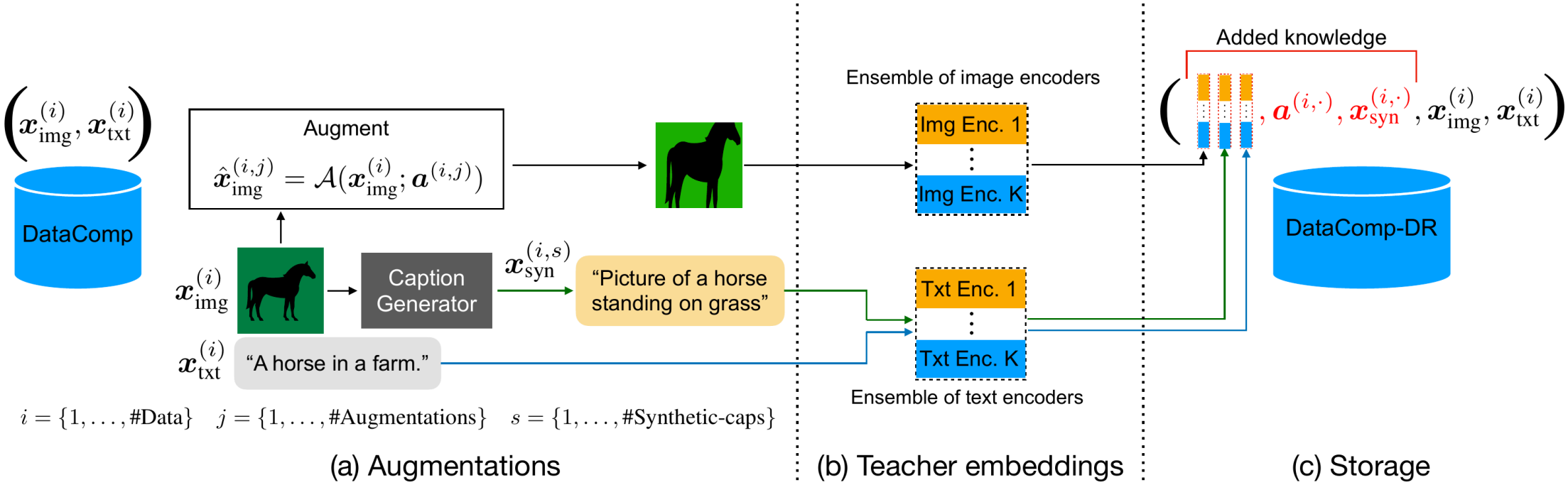
FastViTHD: 1024×1024, 256 tokens

Category	June sales	MoM
Drinks	1,284	+6.2%
Desserts	963	−3.1%
Mains	2,047	+1.8%
Snacks	715	+12.4%

Note: MoM = change from the previous month; June includes 312 online promo orders.
Fictional data, for demonstration only.

The same 256 tokens: for a VLM, **resolution is what matters**

Cost ①: harder compression needs stronger training



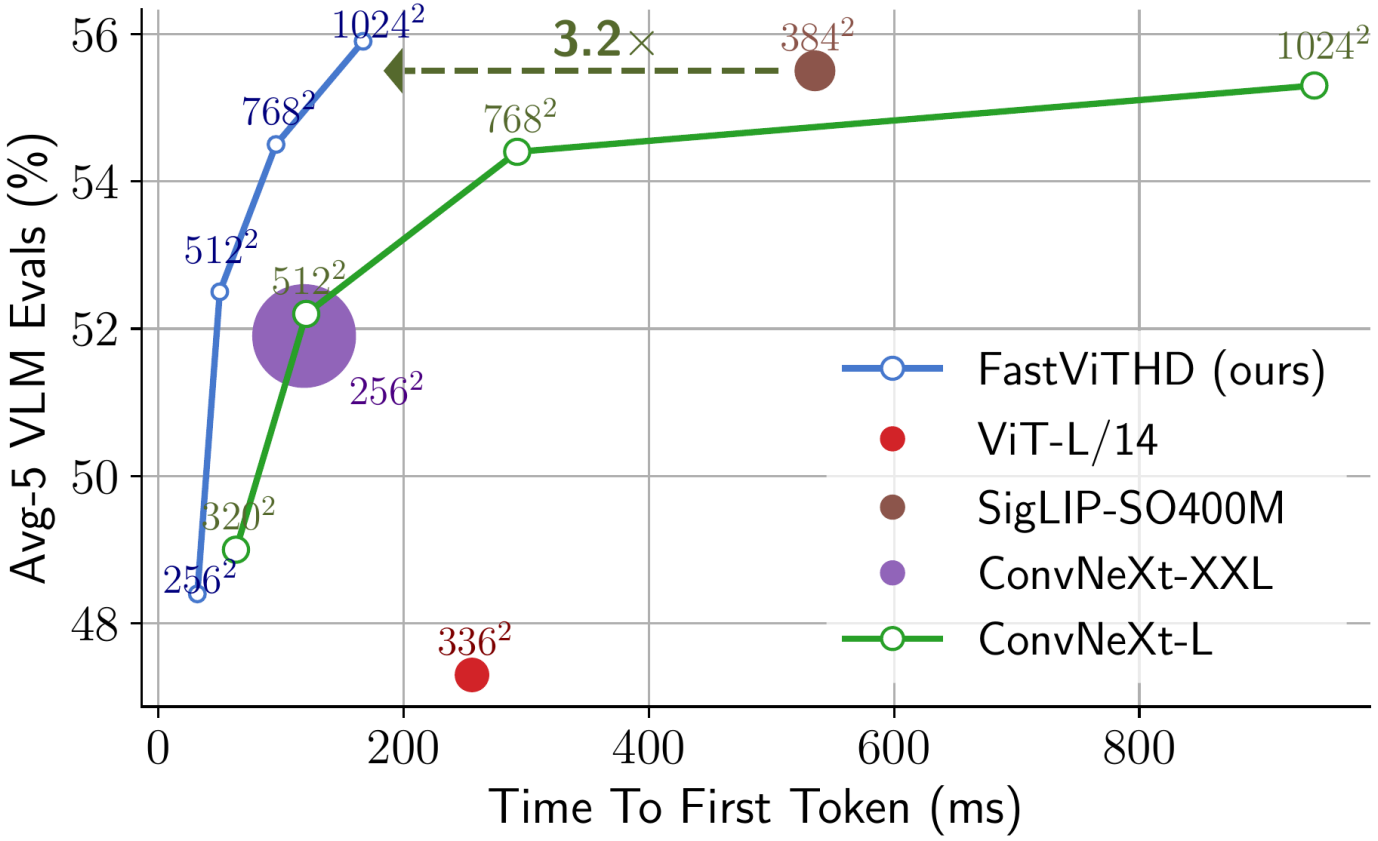
Each token must hold more information → pretraining has to be stronger

Taken from their MobileCLIP: prepare **better captions** and **large teacher models' outputs** for every image-text pair

Cost ②: resolution must match the LLM size

- A very small LLM **can't make use of** ultra-high-resolution images
- However well the vision side compresses, both ends still have to be tuned together

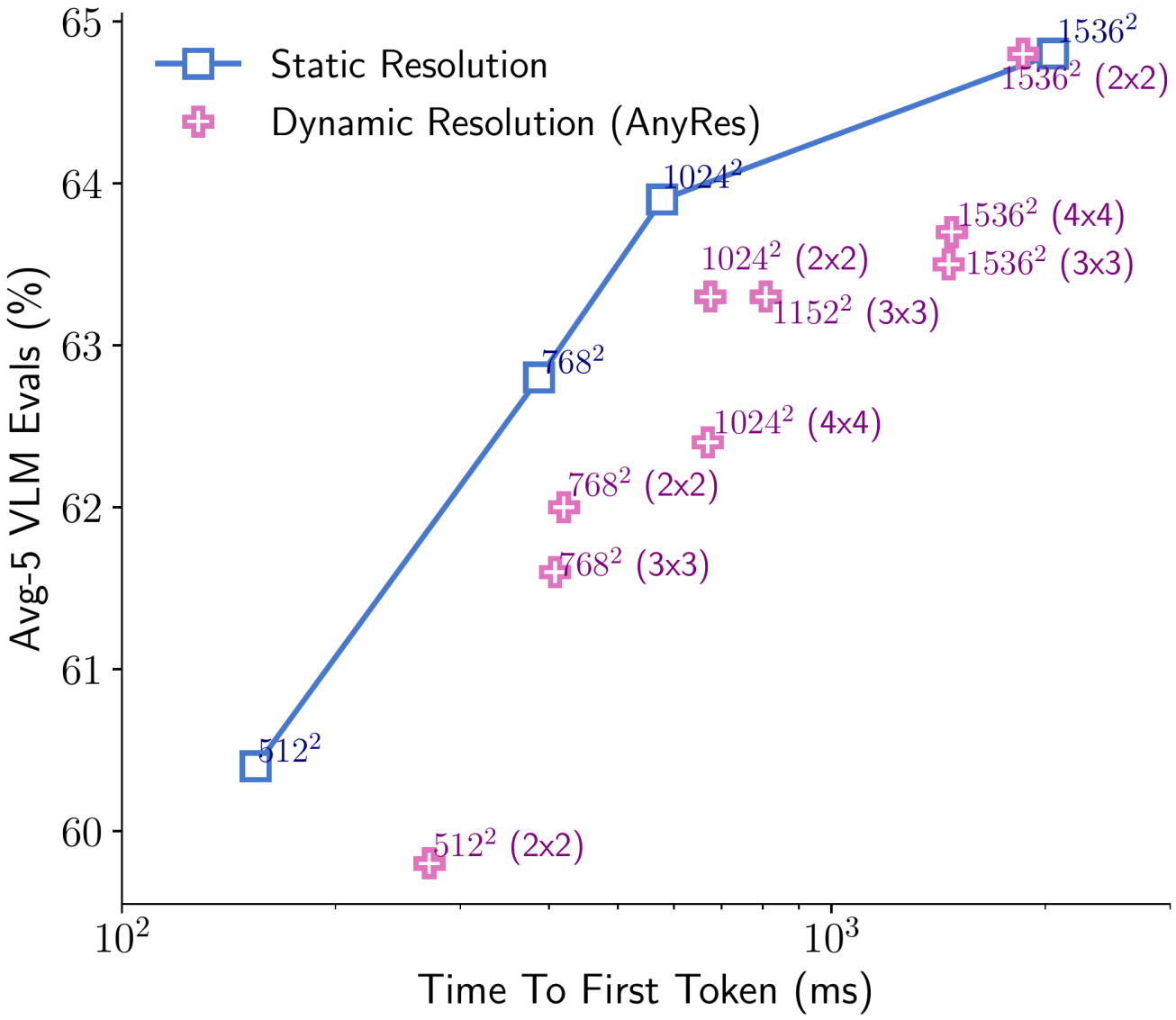
Redesign the encoder from the inside, instead of patching it from the outside



Result ①: same accuracy, 3.2× faster

Same LLM and training; only the vision encoder changes

- Higher = more accurate (average of 5 benchmarks), further left = faster
- The blue line (FastViTHD) sits **up and to the left** of every other encoder
- Even at 1024^2, it is **3.2×** faster than SigLIP at 384^2, with the same accuracy



Result ②: one whole image beats tiles

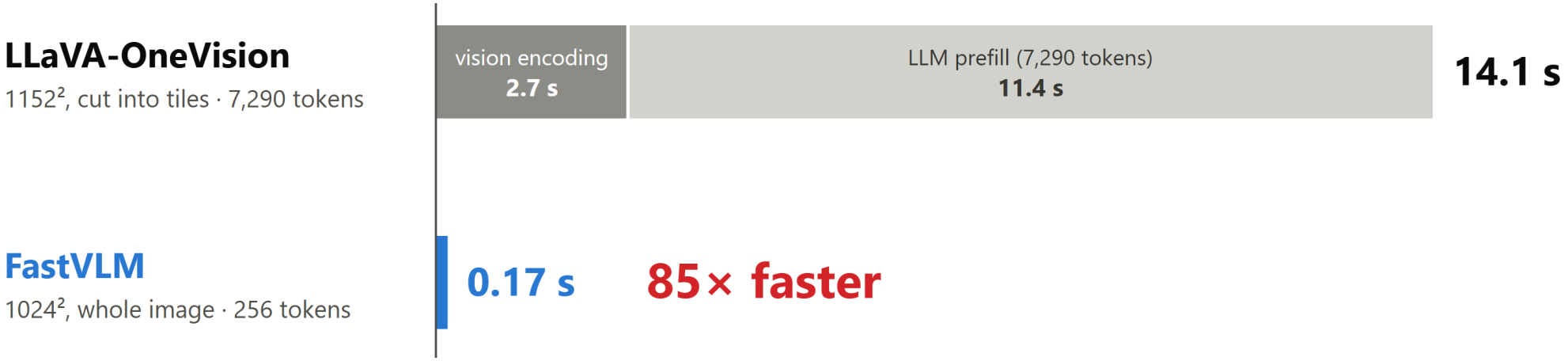
Same model (FastViTHD + 7B LLM), two ways to feed a large image:

- Blue squares: resize the **whole image**
- Pink crosses: cut it into **tiles** (2×2, 3×3, …)

At the same resolution, the whole image scores higher **and** runs faster; tiles only catch up at 1536^2

Result ③: vs. a tiling model with the same LLM

Time to first token, same 0.5B LLM (shorter is faster)



Measured on an M1 Max MacBook Pro (FastVLM paper, Table 6)

256 tokens instead of 7,290 → 85× faster, with scores as good or better

Discussion

What the results show

- The speed-up comes from the encoder: same LLM, same training, only the encoder changed (Result ①)
- High resolution without tiling, few tokens without pruning (Results ② and ③)

What to keep in mind

- Speed was measured on one device (an M1 Max MacBook Pro); other hardware may gain differently
- In Result ③, FastVLM was trained on more data (27.5M vs. 7.7M samples), so not all of the gap is the encoder

→ **Fixing the encoder itself is what makes high resolution affordable**